시작하십시오
Astra Trident

NetApp
January 03, 2023

This PDF was generated from https://docs.netapp.com/ko-kr/trident-2204/trident-get-started/quickstart.html on January 03, 2023. Always check docs.netapp.com for the latest.

목차

시작하십시오 1
시도해 보십시오 1
요구 사항 1
구축 개요 5
Trident 연산자를 사용하여 구축 8
tridentctl을 사용하여 배포합니다 16
다음 단계 19

시작하십시오

시도해 보십시오

NetApp은 사용자가 요청할 수 있는 즉시 사용 가능한 연구소 이미지를 제공합니다 "NetApp 시험 구동". 테스트 드라이브는 3노드 Kubernetes 클러스터 및 Astra Trident가 설치 및 구성된 샌드박스 환경을 제공합니다. Astra Trident에 대해 알아보고 기능을 둘러보는 것도 좋습니다.

또 다른 옵션은 을 보는 것입니다 "kubeadm 설치 가이드" Kubernetes에서 제공:

> 이러한 지침을 운영 환경에 사용하여 구축한 Kubernetes 클러스터를 사용해서는 안 됩니다. 배포 시 제공되는 운영 구축 가이드를 사용하여 즉시 프로덕션할 수 있는 클러스터를 생성할 수 있습니다.

Kubernetes를 처음 사용하는 경우 개념 및 툴에 대해 자세히 알아보십시오 "여기".

요구 사항

지원되는 프런트엔드, 백엔드 및 호스트 구성을 검토하여 시작하십시오.

> Astra Trident가 사용하는 포트에 대한 자세한 내용은 를 참조하십시오 "여기".

지원되는 프런트엔드(오케스트레이터)

Astra Trident는 다음을 비롯한 여러 컨테이너 엔진과 오케스트레이터가 지원됩니다.

- Anthos On-Premise(VMware) 및 Anthos의 Bare Metal 1.9, 1.10, 1.11
- Kubernetes 1.18-1.24
- Mirantis Kubernetes 엔진 3.4
- OpenShift 4.7, 4.8, 4.9, 4.10

Trident 연산자는 다음 릴리즈에서 지원됩니다.

- Anthos On-Premise(VMware) 및 Anthos의 Bare Metal 1.9, 1.10, 1.11
- Kubernetes 1.18-1.24
- OpenShift 4.7, 4.8, 4.9, 4.10

Astra Trident는 GKE(Google Kubernetes Engine), EKS(Amazon Elastic Kubernetes Service), AKS(Azure Kubernetes Service), Rancher, VMware Tanzu Portfolio를 비롯한 기타 완전 관리형 및 자체 관리 Kubernetes 오퍼링과도 작동합니다.

지원되는 백엔드(스토리지)

Astra Trident를 사용하려면 다음 중 하나 이상의 지원되는 백엔드가 필요합니다.

- NetApp ONTAP용 Amazon FSx
- Azure NetApp Files

- Astra 데이터 저장소
- Cloud Volumes ONTAP
- GCP용 Cloud Volumes Service
- FAS/AFF/9.3 이상을 선택합니다
- NetApp All SAN 어레이(ASA)
- NetApp HCI/Element 소프트웨어 11 이상

피처 요구 사항

아래 표에는 Astra Trident의 이번 릴리즈와 함께 사용할 수 있는 기능과 지원하는 Kubernetes 버전이 요약되어 있습니다.

피처	**Kubernetes** 버전	기능 게이트가 필요합니까**?**
CSI Trident	1.18-1.24	아니요
볼륨 스냅샷	1.18-1.24	아니요
체적 스냅샷의 PVC	1.18-1.24	아니요
iSCSI PV 크기 조정	1.18-1.24	아니요
ONTAP 양방향 CHAP	1.18-1.24	아니요
동적 내보내기 정책	1.18-1.24	아니요
Trident 연산자	1.18-1.24	아니요
Auto Worker Node Prep(베타)	1.18-1.24	아니요
CSI 토폴로지	1.18-1.24	아니요

호스트 운영 체제를 테스트했습니다

기본적으로 Astra Trident는 컨테이너에서 실행되므로 모든 Linux 작업자에서 실행됩니다. 그러나 이러한 작업자들은 표준 NFS 클라이언트 또는 iSCSI 이니시에이터를 사용하여 Astra Trident가 제공하는 볼륨을 사용 중인 백엔드에 따라 마운트할 수 있어야 합니다.

Astra Trident는 공식적으로 특정 운영 체제를 "지원"하지 않지만 다음과 같은 Linux 배포판이 작동하는 것으로 알려져 있습니다.

- OpenShift Container Platform에서 지원하는 RedHat CoreOS(RHCOS) 버전
- RHEL 또는 CentOS 7.4 이상
- Ubuntu 18.04 이상

트리덴트틀(tridentctl) 유틸리티도 이 리눅스 배포판에서 실행됩니다.

호스트 구성

사용 중인 백엔드에 따라 NFS 및/또는 iSCSI 유틸리티를 클러스터의 모든 작업자에 설치해야 합니다. 을 참조하십시오 "여기" 를 참조하십시오.

스토리지 시스템 구성

Astra Trident는 백엔드 구성에서 사용하기 전에 스토리지 시스템을 일부 변경해야 할 수 있습니다. 을 참조하십시오 "여기" 를 참조하십시오.

컨테이너 이미지 및 해당 **Kubernetes** 버전

공기 박형 설치의 경우 다음 목록은 Astra Trident를 설치하는 데 필요한 컨테이너 이미지의 참조입니다. tridentctl images 명령을 사용하여 필요한 컨테이너 영상의 목록을 확인합니다.

Kubernetes 버전	컨테이너 이미지
v1.18.0	• NetApp/트리덴트: 22.04.0 • NetApp/트리덴트 - 자동 지원: 22.04 • K8s.GCR.IO/SIG-스토리지/CSI-공급자:v2.2.2 • K8s.GCR.IO/SIG-storage/CSI-attacher:v3.4.0 • K8s.GCR.IO/SIG-storage/CSI-resizer: v1.4.0 • K8s.GCR.IO/SIG-storage/CSI-snapshotter:v3.0.3 • K8s.GCR.IO/SIG-storage/CSI-node-driver-registrar: v2.5.0 • NetApp/트리덴트 - 운영자: 22.04.0(선택 사항)
v1.19.0	• NetApp/트리덴트: 22.04.0 • NetApp/트리덴트 - 자동 지원: 22.04 • K8s.GCR.IO/SIG-스토리지/CSI-공급자:v2.2.2 • K8s.GCR.IO/SIG-storage/CSI-attacher:v3.4.0 • K8s.GCR.IO/SIG-storage/CSI-resizer: v1.4.0 • K8s.GCR.IO/SIG-storage/CSI-snapshotter:v3.0.3 • K8s.GCR.IO/SIG-storage/CSI-node-driver-registrar: v2.5.0 • NetApp/트리덴트 - 운영자: 22.04.0(선택 사항)

Kubernetes 버전	컨테이너 이미지
v1.20.0	• NetApp/트리덴트: 22.04.0 • NetApp/트리덴트 - 자동 지원: 22.04 • K8s.GCR.IO/SIG-스토리지/CSI-공급자:v3.1.0 • K8s.GCR.IO/SIG-storage/CSI-attacher:v3.4.0 • K8s.GCR.IO/SIG-storage/CSI-resizer: v1.4.0 • K8s.GCR.IO/SIG-storage/CSI-snapshotter:v5.0.1 • K8s.GCR.IO/SIG-storage/CSI-node-driver-registrar: v2.5.0 • NetApp/트리덴트 - 운영자: 22.04.0(선택 사항)
v1.21.0	• NetApp/트리덴트: 22.04.0 • NetApp/트리덴트 - 자동 지원: 22.04 • K8s.GCR.IO/SIG-스토리지/CSI-공급자:v3.1.0 • K8s.GCR.IO/SIG-storage/CSI-attacher:v3.4.0 • K8s.GCR.IO/SIG-storage/CSI-resizer: v1.4.0 • K8s.GCR.IO/SIG-storage/CSI-snapshotter:v5.0.1 • K8s.GCR.IO/SIG-storage/CSI-node-driver-registrar: v2.5.0 • NetApp/트리덴트 - 운영자: 22.04.0(선택 사항)
v1.22.0	• NetApp/트리덴트: 22.04.0 • NetApp/트리덴트 - 자동 지원: 22.04 • K8s.GCR.IO/SIG-스토리지/CSI-공급자:v3.1.0 • K8s.GCR.IO/SIG-storage/CSI-attacher:v3.4.0 • K8s.GCR.IO/SIG-storage/CSI-resizer: v1.4.0 • K8s.GCR.IO/SIG-storage/CSI-snapshotter:v5.0.1 • K8s.GCR.IO/SIG-storage/CSI-node-driver-registrar: v2.5.0 • NetApp/트리덴트 - 운영자: 22.04.0(선택 사항)

Kubernetes 버전	컨테이너 이미지
v1.23.0	• NetApp/트리덴트: 22.04.0 • NetApp/트리덴트 - 자동 지원: 22.04 • K8s.GCR.IO/SIG-스토리지/CSI-공급자:v3.1.0 • K8s.GCR.IO/SIG-storage/CSI-attacher:v3.4.0 • K8s.GCR.IO/SIG-storage/CSI-resizer: v1.4.0 • K8s.GCR.IO/SIG-storage/CSI-snapshotter:v5.0.1 • K8s.GCR.IO/SIG-storage/CSI-node-driver-registrar: v2.5.0 • NetApp/트리덴트 - 운영자: 22.04.0(선택 사항)

Kubernetes 버전 1.20 이상에서는 검증된 k8s.gcr.io/sig-storage/scsi-snapshotter:v5.x 이미지를 사용하십시오. v1 버전이 'volumesnapshots.snapshots.storage.k8s.io' CRD를 지원하는 경우에만 해당됩니다. v1beta1 버전이 v1 버전 없이 CRD를 지원하는 경우 검증된 k8s.gcr.io/sig-storage/scsi-snapshotter:v3.x 이미지를 사용하십시오.

구축 개요

Trident 운영자나 tridentctl을 사용하여 Astra Trident를 배포할 수 있습니다.

22.04 릴리즈부터는 Astra Trident가 설치될 때마다 AES 키가 더 이상 다시 생성되지 않습니다. 이 릴리즈를 통해 Astra Trident는 설치 전반에 걸쳐 유지되는 새로운 비밀 객체를 설치합니다. 즉, 22.04의 tridentctl은 Trident의 이전 버전을 제거할 수 있지만 이전 버전에서는 22.04 설치를 제거할 수 없습니다.

배포 방법을 선택합니다

사용할 배포 방법을 결정하려면 다음 사항을 고려하십시오.

Trident 연산자를 사용해야 하는 이유는 무엇입니까**?**

를 클릭합니다 "Trident 운영자" Astra Trident 리소스를 동적으로 관리하고 설정 단계를 자동화하는 훌륭한 방법입니다. 몇 가지 전제 조건이 충족되어야 합니다. 을 참조하십시오 "설명합니다".

Trident 운영자는 아래에 설명된 여러 가지 이점을 제공합니다.

자동 복구 기능

Astra Trident 설치를 모니터링하고 구축이 삭제되거나 실수로 수정된 경우와 같은 문제를 해결하기 위한 조치를 적극적으로 취할 수 있습니다. 운용자가 deployment로 설정된 경우 trident-operator-<generated-id>'pod가 생성된다. 이 포드는 아스트라 트리덴트 설치와 트리젠오케스트레이터 CR을 연결하며 항상 하나의 활성 '트리젠터레이터'만 존재하도록 합니다. 즉, 운영자는 클러스터에 Astra Trident 인스턴스가 하나만 있고 설정을 제어하여 설치가 매우 강력한지 확인합니다. 설치 변경(예: 배포 또는 노드 반점 삭제)이 수행되면 운영자가 이를 식별하고 개별적으로 수정합니다.

기존 설치에 대한 손쉬운 업데이트

기존 배포를 운영자로 쉽게 업데이트할 수 있습니다. 설치에 대한 업데이트를 하기 위해 '트리엔오케스트레이터' CR만 편집하면 됩니다. 예를 들어, Astra Trident를 활성화하여 디버그 로그를 생성해야 하는 시나리오를 생각해 보십시오.

이렇게 하려면 '트리엔오케스트레이터'를 패치하여 'pec.debug'를 'true'로 설정합니다.

```
kubectl patch torc <trident-orchestrator-name> -n trident --type=merge -p
'{"spec":{"debug":true}}'
```

'트리엔오케스트레이터'가 업데이트된 후, 운용자는 기존 설치 시 업데이트 및 패치를 처리한다. 이렇게 하면 새 포드가 생성되어 설치를 적절하게 수정할 수 있습니다.

Kubernetes 업그레이드 자동 처리

클러스터의 Kubernetes 버전이 지원되는 버전으로 업그레이드되면 운영자는 기존 Astra Trident 설치를 자동으로 업데이트하고 Kubernetes 버전 요구사항을 충족하도록 변경합니다.

> 클러스터가 지원되지 않는 버전으로 업그레이드되면 운영자는 Astra Trident를 설치할 수 없습니다. Astra Trident가 운영자와 함께 이미 설치된 경우 Astra Trident가 지원되지 않는 Kubernetes 버전에 설치되었음을 나타내는 경고가 표시됩니다.

Cloud Manager를 사용하여 Kubernetes 클러스터를 관리합니다

와 함께 "Cloud Manager를 사용하는 Astra Trident"Astra Trident의 최신 버전으로 업그레이드하고, 스토리지 클래스를 추가 및 관리하고, 작업 환경에 연결한 다음, Cloud Backup Service를 사용하여 영구 볼륨을 백업할 수 있습니다. Cloud Manager는 Trident 연산자를 사용하여 수동으로 또는 Helm을 사용하여 Astra Trident 구축을 지원합니다.

Helm을 사용해야 하는 이유는 무엇입니까?

Astra Trident 21.01부터 Helm을 사용하여 관리하는 다른 애플리케이션이 있는 경우 Helm을 사용하여 배포를 관리할 수도 있습니다.

언제 사용해야 하는 지 `tridenctl`?

업그레이드해야 하는 기존 배포가 있거나 배포를 고도로 사용자 지정하려는 경우 를 사용하는 방법을 고려해야 합니다 "tridentctl 을 선택합니다". Astra Trident를 구축하는 기존 방법입니다.

배포 방법 간에 이동할 때의 고려 사항

배포 방법 간에 이동해야 하는 시나리오를 상상하기란 쉽지 않습니다. "tridentctl" 배포에서 연산자 기반 배포로 이동하거나 그 반대로 이동하려면 다음 사항을 고려해야 합니다.

- Astra Trident를 제거할 때는 항상 동일한 방법을 사용하십시오. tridentctl을 사용하여 배포한 경우에는 해당 버전의 tridentctl 바이너리를 사용하여 Astra Trident를 제거해야 합니다. 마찬가지로 운영자와 함께 배포하는 경우 '트리젠오케스트레이터' CR을 편집하고 '스펙트 제거=true'를 설정하여 Astra Trident를 제거해야 합니다.
- Astra Trident를 배포하기 위해 "tridentctl"을 사용하여 제거할 운영자 기반 배포가 있는 경우 먼저 'TridentOrchestrator'를 편집하고 'csec.uninstall=true'를 설정하여 Astra Trident를 제거해야 합니다. 그런 다음 '트리엔오케스트레이터'와 운영자 구축을 삭제합니다. 그런 다음 tridentctl을 사용하여 설치할 수 있습니다.

- 작업자 기반의 수동 배포를 사용하고 H제어 기반 Trident 연산자 배포를 사용하려는 경우 먼저 수동으로 연산자를 제거한 다음 Helm 설치를 수행해야 합니다. 이를 통해 Helm은 필요한 레이블 및 주석을 사용하여 Trident 연산자를 배포할 수 있습니다. 이렇게 하지 않으면 레이블 유효성 검사 오류 및 주석 유효성 검사 오류와 함께 H제어 기반 Trident 연산자 배포가 실패합니다. "tridentctl" 기반 배포를 사용하는 경우 문제 없이 Helm 기반 배포를 사용할 수 있습니다.

배포 모드를 이해합니다

Astra Trident를 구축하는 방법에는 세 가지가 있습니다.

Standard 구축

Kubernetes 클러스터에서 Trident를 구축하면 Astra Trident 설치 관리자가 다음 두 가지 작업을 수행할 수 있습니다.

- 인터넷을 통해 컨테이너 이미지를 가져오는 중입니다
- Kubernetes 클러스터의 모든 유효한 노드에서 Astra Trident Pod를 가동하는 구축 및/또는 노드 데모 생성

이와 같은 표준 배포는 두 가지 방법으로 수행할 수 있습니다.

- tridentctl install 사용
- Trident 연산자 사용 Trident 연산자는 수동으로 또는 Helm을 사용하여 배포할 수 있습니다.

이 설치 모드는 Astra Trident를 설치하는 가장 쉬운 방법이며 네트워크 제한이 없는 대부분의 환경에서 작동합니다.

오프라인 배포

"tridentctl install"을 호출할 때 '--image-registry" 플래그를 사용하여 개인 이미지 레지스트리를 가리킬 수 있습니다. Trident 연산자를 사용하여 배포하는 경우 ' Trident' 오케스트레이터에서 's.imageRegistry'를 지정할 수도 있습니다. 이 레지스트리에는 가 포함되어 있어야 합니다 "Trident 이미지", "Trident AutoSupport 이미지"및 Kubernetes 버전에서 요구하는 CSI 사이드카 이미지를 참조하십시오.

배포를 사용자 지정하려면 tridentctl 을 사용하여 Trident의 리소스에 대한 매니페스트를 생성할 수 있습니다. 여기에는 구축, 개발/제거, 서비스 계정, Astra Trident가 설치의 일부로 생성한 클러스터 역할 등이 포함됩니다.

배포 사용자 지정에 대한 자세한 내용은 다음 링크를 참조하십시오.

- "운영자 기반 배포를 사용자 지정합니다"
- "85cda507040c129ee606d4d8df583b90"

> 개인 이미지 리포지토리를 사용하는 경우 개인 레지스트리 URL 끝에 /sig-storage를 추가해야 합니다. tridentctl 배포를 위해 개인 레지스트리를 사용할 때는 '--trident-image' 및 '--autosupport-image'를 '--image-registry'와 함께 사용해야 합니다. Trident 연산자를 사용하여 Astra Trident를 배포하는 경우 Orchestrator CR에 설치 매개변수에 "tridentImage" 및 "autostsupportImage"가 포함되어 있는지 확인합니다.

원격 배포

다음은 원격 배포 프로세스에 대한 상위 수준의 개요입니다.

- Astra Trident를 구축하려는 원격 머신에 적절한 버전의 kubeck tl을 배포합니다.

- Kubernetes 클러스터에서 구성 파일을 복사하고 원격 시스템에서 'KUBECONFIG' 환경 변수를 설정합니다.
- 필요한 Kubernetes 클러스터에 연결할 수 있는지 확인하려면 "kubbtl get nodes" 명령을 시작합니다.
- 표준 설치 단계를 사용하여 원격 컴퓨터에서 배포를 완료합니다.

기타 알려진 구성 옵션

VMware Tanzu 포트폴리오 제품에 Astra Trident를 설치할 경우:

- 클러스터는 권한이 있는 워크로드를 지원해야 합니다.
- kubelet-dir 플래그는 kubelet 디렉토리의 위치로 설정해야 합니다. 기본적으로 이 값은 '/var/vcap/data/kubelet'입니다.

  Trident 연산자, Hrom 및 tridentctl 배포에서는 -kubelet -dir 을 사용하여 kubelet 위치를 지정하는 작업이 알려져 있습니다.

Trident 연산자를 사용하여 구축

Trident 연산자를 사용하여 Astra Trident를 배포할 수 있습니다. Trident 연산자는 다음 두 가지 방법 중 하나로 배포할 수 있습니다.

- Trident 사용 "Helm 차트": 제어 도표는 Trident 연산자를 배포하고 Trident를 한 번에 설치합니다.
- 수동: Trident에서 를 제공합니다 "Bundle.YAML" 연산자를 설치하고 관련 개체를 만드는 데 사용할 수 있는 파일입니다.

에 아직 익숙하지 않은 경우 "기본 개념"이제 아주 좋은 시간입니다.

필요한 것

Astra Trident를 구축하려면 다음과 같은 사전 요구 사항을 충족해야 합니다.

- Kubernetes 1.18-1.24를 실행하는 지원되는 Kubernetes 클러스터에 대한 모든 권한이 있습니다.
- 지원되는 NetApp 스토리지 시스템에 액세스할 수 있습니다.
- 모든 Kubernetes 작업자 노드에서 볼륨을 마운트할 수 있습니다.
- 사용할 Kubernetes 클러스터를 관리하도록 "kubbtl"(또는 OpenShift를 사용하는 경우 OC)가 설치 및 구성된 Linux 호스트를 보유하고 있습니다.
- Kubernetes 클러스터 구성을 가리키도록 'KUBECONFIG' 환경 변수를 설정했습니다.
- 을(를) 활성화했습니다 "Astra Trident에서 요구하는 기능 게이트".
- Docker Enterprise와 함께 Kubernetes를 사용하는 경우, "다음 단계에 따라 CLI 액세스를 설정합니다".

다 잡았나요? 좋습니다! 그럼 이제 시작하겠습니다.

Trident 연산자를 구축하고 Hrom을 사용하여 Astra Trident를 설치합니다

나열된 단계를 수행하여 Helm을 사용하여 Trident 연산자를 배포합니다.

필요한 것

위에 나열된 필수 구성 요소 외에도 Hrom을 사용하여 Trident 연산자를 구축하려면 다음이 필요합니다.

- Kubernetes 1.18-1.24
- Helm 버전 3

단계

1. Trident의 제어 리포지토리 추가:

```
helm repo add netapp-trident https://netapp.github.io/trident-helm-chart
```

2. 'helm install' 명령을 사용하여 배포 이름을 지정합니다. 다음 예를 참조하십시오.

```
helm install <release-name> netapp-trident/trident-operator --version
22.4.0 --namespace <trident-namespace>
```

> Trident의 네임스페이스를 아직 만들지 않은 경우 '--create-namespace' 매개 변수를 'helm install' 명령에 추가할 수 있습니다. 그런 다음 Helm이 자동으로 네임스페이스를 만듭니다.

설치 중에 구성 데이터를 전달하는 방법에는 두 가지가 있습니다.

- '--Values'(또는 '-f'): 재정의가 있는 YAML 파일을 지정합니다. 이 옵션은 여러 번 지정할 수 있으며 가장 오른쪽 파일이 우선 적용됩니다.
- '--set': 명령행에서 overrides를 지정합니다.

예를 들어, 기본값인 debug를 변경하려면 다음 '--set' 명령을 실행합니다.

```
$ helm install <name> netapp-trident/trident-operator --version 22.4.0
--set tridentDebug=true
```

제어 차트의 일부인 Values.YAML 파일은 키 목록과 기본값을 제공합니다.

'Helm list'는 이름, 네임스페이스, 차트, 상태, 앱 버전, 개정 번호 등

Trident 연산자를 수동으로 배포합니다

나열된 단계를 수행하여 Trident 연산자를 수동으로 배포합니다.

1단계: Kubernetes 클러스터 검증

가장 먼저 해야 할 일은 Linux 호스트에 로그인하여 해당 호스트가 관리 중인 _ 작업 _ 을(를) 관리하고 있는지 확인하는 것입니다. "지원되는 Kubernetes 클러스터" 에 필요한 권한이 있어야 합니다.

> OpenShift에서는 뒤에 나오는 모든 예시에서 kubtl 대신 OC를 사용하고, OC login-u system:admin 또는 OC login-u kubbe-admin을 실행하여 먼저 * system:admin * 으로 로그인합니다.

Kubernetes 버전을 확인하려면 다음 명령을 실행합니다.

```
kubectl version
```

Kubernetes 클러스터 관리자 권한이 있는지 확인하려면 다음 명령을 실행합니다.

```
kubectl auth can-i '*' '*' --all-namespaces
```

Docker Hub에서 이미지를 사용하는 Pod를 시작하고 Pod 네트워크를 통해 스토리지 시스템에 연결할 수 있는지 확인하려면 다음 명령을 실행합니다.

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

2단계**:** 운영자를 다운로드하고 설정합니다

> 21.01부터 Trident 연산자는 클러스터 범위입니다. Trident 연산자를 사용하여 Trident를 설치하려면 "트리엔오케스트레이터" CRD(사용자 정의 리소스 정의)를 생성하고 다른 리소스를 정의해야 합니다. Astra Trident를 설치하기 전에 다음 단계를 수행하여 운영자를 설정해야 합니다.

1. 의 최신 버전을 다운로드합니다 "Trident 설치 프로그램 번들" 다운로드 섹션에서 압축을 풉니다.

   ```
   wget
   https://github.com/NetApp/trident/releases/download/v22.04.0/trident-
   installer-22.04.0.tar.gz
   tar -xf trident-installer-22.04.0.tar.gz
   cd trident-installer
   ```

2. 적절한 CRD 매니페스트를 사용하여 '트리엔오케스트레이터' CRD를 생성합니다. 그런 다음 나중에 운영자가 설치를 인스턴스화하기 위해 '트리엔오케스트레이터' 사용자 정의 리소스를 만듭니다.

   다음 명령을 실행합니다.

   ```
   kubectl create -f
   deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
   ```

3. '트리엔오케스트레이터' CRD가 생성된 후 운용자 배치에 필요한 다음과 같은 자원을 생성한다.

   - 연산자를 위한 ServiceAccount입니다
   - ServiceAccount에 대한 ClusterRole 및 ClusterRoleBinding
   - 전용 PodSecurityPolicy

- 작업자 자체

  Trident 설치 프로그램에는 이러한 리소스를 정의하는 매니페스트가 포함되어 있습니다. 기본적으로 연산자는 '삼원' 네임스페이스에 배포됩니다. 트리덴트 네임스페이스가 없으면 다음 매니페스트를 사용하여 만듭니다.

```
$ kubectl apply -f deploy/namespace.yaml
```

4. 기본 '삼중류' 네임스페이스 이외의 이름 공간에 운용자를 배치하려면 '세르이스계정.YAML', '클러스터구속력.YAML', 'operator.YAML'을 업데이트하고 'bundle.YAML'을 작성해야 합니다.

   다음 명령을 실행하여 YAML 매니페스트를 업데이트하고 '고객.YAML'을 사용하여 'bundle.YAML'을 생성합니다.

```
kubectl kustomize deploy/ > deploy/bundle.yaml
```

   다음 명령을 실행하여 리소스를 생성하고 연산자를 배포합니다.

```
kubectl create -f deploy/bundle.yaml
```

5. 배치한 후 작업자의 상태를 확인하려면 다음을 수행합니다.

```
$ kubectl get deployment -n <operator-namespace>
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator    1/1     1            1           3m

$ kubectl get pods -n <operator-namespace>
NAME                                READY   STATUS             RESTARTS
AGE
trident-operator-54cb664d-lnjxh     1/1     Running            0
3m
```

운영자 배포는 클러스터의 작업자 노드 중 하나에서 실행되고 있는 포드를 성공적으로 생성합니다.

> Kubernetes 클러스터에는 운영자의 인스턴스 * 하나가 있어야 합니다. Trident 연산자의 여러 배포를 생성하지 마십시오.

3단계: 작성 `TridentOrchestrator` **Trident**를 설치합니다

이제 연산자를 사용하여 Astra Trident를 설치할 준비가 되었습니다! 이 작업을 수행하려면 '트리엔오케스트레이터'를 만들어야 합니다. Trident 설치 프로그램에는 'Trident Orchestrator'를 만들기 위한 예제 정의가 포함되어 있습니다. 이렇게 하면 '삼중덴트' 네임스페이스에서 설치가 시작됩니다.

```
$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created

$ kubectl describe torc trident
Name:        trident
Namespace:
Labels:      <none>
Annotations: <none>
API Version: trident.netapp.io/v1
Kind:        TridentOrchestrator
...
Spec:
  Debug:     true
  Namespace: trident
Status:
  Current Installation Params:
    IPv6:                      false
    Autosupport Hostname:
    Autosupport Image:         netapp/trident-autosupport:21.04
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                     true
    Enable Node Prep:          false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:21.04.0
  Message:                  Trident installed  Namespace:
trident
  Status:                   Installed
  Version:                  v21.04.0
Events:
    Type Reason Age From Message ---- ------ ---- ---- -------Normal
    Installing 74s trident-operator.netapp.io Installing Trident Normal
    Installed 67s trident-operator.netapp.io Trident installed
```

Trident 연산자를 사용하면 ' Trident' 사양의 특성을 사용하여 Astra Trident가 설치되는 방식을 사용자 지정할 수 있습니다. 을 참조하십시오 "Trident 구축을 사용자 지정합니다".

'트리엔오케스트레이터'의 상태는 설치가 성공적이었음을 나타내며 설치된 Trident의 버전을 표시합니다.

상태	설명
설치 중	운영자는 이 '트리엔오케스트레이터' CR을 사용하여 Astra Trident를 설치하고 있습니다.
설치되어 있습니다	Astra Trident가 성공적으로 설치되었습니다.
제거 중	이 경우, 'pec.uninstall=true'로 인해 운용자가 Astra Trident를 설치 제거 중입니다.
제거되었습니다	Astra Trident가 제거되었습니다.
실패했습니다	운영자가 Astra Trident를 설치, 패치, 업데이트 또는 제거할 수 없습니다. 이 상태에서 자동으로 복구를 시도합니다. 이 상태가 지속되면 문제 해결이 필요합니다.
업데이트 중	운영자가 기존 설치를 업데이트하고 있습니다.
오류	트리젠터레이터(트리젠터레이터)는 사용하지 않는다. 다른 파일이 이미 있습니다.

설치 과정에서 트리엔오케스트레이터 상태가 설치 상태에서 설치 상태로 변경됩니다. 'Failed(실패)' 상태를 확인하고 운용자가 자체적으로 복구할 수 없는 경우 운용자의 로그를 확인해야 한다. 를 참조하십시오 "문제 해결" 섹션을 참조하십시오.

생성된 포드를 살펴보고 Astra Trident 설치가 완료되었는지 확인할 수 있습니다.

```
$ kubectl get pod -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-7d466bf5c7-v4cpw        5/5     Running   0           1m
trident-csi-mr6zc                   2/2     Running   0           1m
trident-csi-xrp7w                   2/2     Running   0           1m
trident-csi-zh2jt                   2/2     Running   0           1m
trident-operator-766f7b8658-ldzsv   1/1     Running   0           3m
```

또한 'tridentctl'을 사용하여 설치된 Astra Trident의 버전을 확인할 수도 있습니다.

```
$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.04.0        | 21.04.0        |
+----------------+----------------+
```

이제 백엔드를 생성할 수 있습니다. 을 참조하십시오 "구축 후 작업".

배포 중 문제 해결에 대한 자세한 내용은 을 참조하십시오 "문제 해결" 섹션을 참조하십시오.

Trident 운영자 배포를 사용자 지정합니다

Trident 연산자를 사용하면 ' Trident' 사양의 특성을 사용하여 Astra Trident가 설치되는 방식을 사용자 지정할 수 있습니다.

속성 목록은 다음 표를 참조하십시오.

매개 변수	설명	기본값
'네임스페이스'	Astra Trident를 설치할 네임스페이스입니다	"기본값"
"버그"	Astra Trident에 대한 디버깅을 활성화합니다	거짓
'IPv6'	IPv6를 통해 Astra Trident를 설치합니다	거짓
k8sTimeout	Kubernetes 작업 시간이 초과되었습니다	30초
이제 자동 지원입니다	AutoSupport 번들을 NetApp에 자동으로 보내지 않습니다	거짓
"enableNodePrep"입니다	작업자 노드 종속성 자동 관리(* beta*)	거짓
자동 지원 이미지	AutoSupport 텔레메트리 컨테이너 이미지입니다	"NetApp/트리덴트 - 자동 지원: 21.04.0"
자동 지원 프록시	AutoSupport 텔레메트리 전송을 위한 프록시의 주소/포트입니다	"<a href="http://proxy.example.com:8888"" class="bare">http://proxy.example.com:8888"</a>
"제거"를 선택합니다	Astra Trident를 제거하는 데 사용되는 플래그입니다	거짓
로그포맷	사용할 Astra Trident 로깅 형식[text,json]	"텍스트"
트리덴티이미지	설치할 Astra Trident 이미지	"NetApp/트리덴트: 21.04"
'imageRegistry'입니다	'<registry FQDN>[:port][/subpath]' 형식의 내부 레지스트리 경로입니다	"k8s.gcr.io/sig-storage(k8s 1.18 이상) 또는 quay.io/k8scsi"
쿠벨레트디렉토리	호스트의 kubelet 디렉토리에 대한 경로입니다	"/var/lib/kubelet"
"위피"	Astra Trident를 완전히 제거하기 위해 삭제할 리소스 목록입니다	
'imagePullSecrets'	내부 레지스트리에서 이미지를 가져올 수 있는 비밀	
'controllerPluginNodeSelector'	Trident 컨트롤러 CSI 플러그인을 실행하는 Pod용 추가 노드 선택기. pod.spec.nodeSelector 과 동일한 형식을 따릅니다.	기본값 없음, 선택 사항

매개 변수	설명	기본값
'컨트롤러 PluginTolerations'	Trident 컨트롤러 CSI 플러그인을 실행하는 Pod의 허용 설정을 재정의합니다. pod.spec.Tolerations와 같은 형식을 따릅니다.	기본값 없음, 선택 사항
노드플러거노드 선택기	Trident Node CSI 플러그인을 실행하는 Pod용 추가 노드 선택기 pod.spec.nodeSelector 과 동일한 형식을 따릅니다.	기본값 없음, 선택 사항
노드 PluginTolerations	Trident Node CSI 플러그인을 실행하는 Pod의 허용 설정을 재정의합니다. pod.spec.Tolerations와 같은 형식을 따릅니다.	기본값 없음, 선택 사항

pec.namespace` 는 어느 네임스페이스 Astra Trident가 설치되어 있는지 나타내기 위해 트리엔터레이터에 지정됩니다. Astra Trident가 설치된 후에는 이 매개 변수 * 를 업데이트할 수 없습니다. 이렇게 하면 트리엔오케스트레이터 상태가 실패로 바뀝니다. Astra Trident는 네임스페이스 간에 마이그레이션할 수 없습니다.

자동 작업자 노드 준비는 비생산 환경에서만 사용할 수 있는 * 베타 기능입니다.

포드 매개 변수 포맷에 대한 자세한 내용은 을 참조하십시오 "노드에 Pod 할당".

'트리엔오케스트레이터'를 정의할 때 위에서 설명한 속성을 사용하여 설치를 사용자 정의할 수 있습니다. 예를 들면 다음과 같습니다.

```
$ cat deploy/crds/tridentorchestrator_cr_imagepullsecrets.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  imagePullSecrets:
  - thisisasecret
```

다음은 노드 선택기를 사용하여 Trident를 구축하는 방법을 보여 주는 또 다른 예입니다.

```
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  controllerPluginNodeSelector:
    nodetype: master
  nodePluginNodeSelector:
    storage: netapp
```

트리벤트오케스트레이터 인수를 통해 설치를 사용자 정의하려는 경우, 필요에 따라 수정할 수 있는 맞춤형 YAML 매니페스트를 생성하기 위해 'tridentctl'을 사용하는 것이 좋습니다.

tridentctl을 사용하여 배포합니다

tridentctl을 사용하여 Astra Trident를 배포할 수 있습니다.

에 아직 익숙하지 않은 경우 "기본 개념"이제 아주 좋은 시간입니다.

배포를 사용자 지정하려면 을 참조하십시오 "여기".

필요한 것

Astra Trident를 구축하려면 다음과 같은 사전 요구 사항을 충족해야 합니다.

- 지원되는 Kubernetes 클러스터에 대한 모든 권한이 있습니다.
- 지원되는 NetApp 스토리지 시스템에 액세스할 수 있습니다.
- 모든 Kubernetes 작업자 노드에서 볼륨을 마운트할 수 있습니다.
- 사용할 Kubernetes 클러스터를 관리하도록 "kubbtl"(또는 OpenShift를 사용하는 경우 OC)가 설치 및 구성된 Linux 호스트를 보유하고 있습니다.
- Kubernetes 클러스터 구성을 가리키도록 'KUBECONFIG' 환경 변수를 설정했습니다.
- 을(를) 활성화했습니다 "Astra Trident에서 요구하는 기능 게이트".
- Docker Enterprise와 함께 Kubernetes를 사용하는 경우, "다음 단계에 따라 CLI 액세스를 설정합니다".

다 잡았나요? 좋습니다! 그럼 이제 시작하겠습니다.

배포 사용자 지정에 대한 자세한 내용은 을 참조하십시오 "여기".

1단계: Kubernetes 클러스터 검증

가장 먼저 해야 할 일은 Linux 호스트에 로그인하여 해당 호스트가 관리 중인 _ 작업 _ 을(를) 관리하고 있는지 확인하는 것입니다. "지원되는 Kubernetes 클러스터" 에 필요한 권한이 있어야 합니다.

OpenShift에서는 다음 모든 예제에서 kubeck 대신 OC를 사용하며 먼저 OC login-u system:admin 또는 OC login-u kubbe-admin을 실행하여 * system:admin * 으로 로그인해야 합니다.

Kubernetes 버전을 확인하려면 다음 명령을 실행합니다.

```
kubectl version
```

Kubernetes 클러스터 관리자 권한이 있는지 확인하려면 다음 명령을 실행합니다.

```
kubectl auth can-i '*' '*' --all-namespaces
```

Docker Hub에서 이미지를 사용하는 Pod를 시작하고 Pod 네트워크를 통해 스토리지 시스템에 연결할 수 있는지 확인하려면 다음 명령을 실행합니다.

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

Kubernetes 서버 버전을 식별하십시오. Astra Trident를 설치할 때 사용합니다.

2단계: 설치 프로그램을 다운로드하고 압축을 풉니다

Trident 설치 관리자는 Trident Pod를 생성하고 상태를 유지하는 데 사용되는 CRD 객체를 구성하며, 클러스터 호스트에 볼륨 프로비저닝 및 연결과 같은 작업을 수행하는 CSI 사이드카를 초기화합니다.

의 최신 버전을 다운로드할 수 있습니다 "Trident 설치 프로그램 번들" 다운로드 섹션에서 압축을 풉니다.

예를 들어, 최신 버전이 21.07.1인 경우:

```
wget https://github.com/NetApp/trident/releases/download/v21.07.1/trident-
installer-21.07.1.tar.gz
tar -xf trident-installer-21.07.1.tar.gz
cd trident-installer
```

3단계: Astra Trident 설치

tridentctl install 명령을 실행하여 원하는 네임스페이스에 Astra Trident를 설치합니다.

```
$ ./tridentctl install -n trident
....
INFO Starting Trident installation.                namespace=trident
INFO Created service account.
INFO Created cluster role.
INFO Created cluster role binding.
INFO Added finalizers to custom resource definitions.
INFO Created Trident service.
INFO Created Trident secret.
INFO Created Trident deployment.
INFO Created Trident daemonset.
INFO Waiting for Trident pod to start.
INFO Trident pod started.                          namespace=trident
pod=trident-csi-679648bd45-cv2mx
INFO Waiting for Trident REST interface.
INFO Trident REST interface is up.                 version=21.07.1
INFO Trident installation succeeded.
....
```

설치 프로그램이 완료되면 다음과 같이 표시됩니다. Kubernetes 클러스터의 노드 수에 따라 다음과 같은 Pod를 더 많이 관찰할 수 있습니다.

```
$ kubectl get pod -n trident
NAME                            READY   STATUS    RESTARTS   AGE
trident-csi-679648bd45-cv2mx    4/4     Running   0          5m29s
trident-csi-vgc8n               2/2     Running   0          5m29s

$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.07.1        | 21.07.1        |
+----------------+----------------+
```

위 예제와 유사한 출력이 표시되는 경우 이 단계를 완료했지만 Astra Trident가 아직 완전히 구성되지 않은 것입니다. 다음 단계를 계속 진행하십시오. 을 참조하십시오 "구축 후 작업".

그러나 설치 프로그램이 성공적으로 완료되지 않거나 * "trident-scsi-<generated id>"를 실행하는 * 가 표시되지 않으면 플랫폼이 설치되지 않은 것입니다.

배포 중 문제 해결에 대한 자세한 내용은 을 참조하십시오 "문제 해결" 섹션을 참조하십시오.

tridentctl 배포를 사용자 지정합니다

Trident 설치 프로그램을 사용하여 특성을 사용자 지정할 수 있습니다. 예를 들어, Trident 이미지를 개인 저장소에 복사한 경우 '--trident-image'를 사용하여 이미지 이름을 지정할 수 있습니다. Trident 이미지와 필요한 CSI 사이드카 이미지를 개인 저장소에 복사한 경우 '<registry FQDN>[:port]' 형식을 사용하는 '--image-registry' 스위치를 사용하여 해당 리포지토리의 위치를 지정하는 것이 좋습니다.

Astra Trident가 작업자 노드를 자동으로 구성하도록 하려면 '--enable-node-prep'를 사용합니다. 작동 방식에 대한 자세한 내용은 을 참조하십시오 "여기".

> 자동 작업자 노드 준비는 비생산 환경에서만 사용할 수 있는 * 베타 기능입니다.

kubelet이 평소 하던 /var/lib/kubelet가 아닌 다른 경로에 데이터를 보관하는 Kubernetes 배포를 사용하는 경우 -kubelet-dir을 사용하여 대체 경로를 지정할 수 있습니다.

설치 관리자의 인수 이외에 설치를 사용자 지정해야 하는 경우 배포 파일을 사용자 지정할 수도 있습니다. '--generate-custom-YAML' 파라미터를 사용하면 설치 프로그램의 '셋업' 디렉토리에 다음과 같은 YAML 파일이 생성됩니다.

- 트리덴트-클러스터롤리바인딩.YAML
- '트라이덴트 배포.YAML'
- 트리덴트-CRD.YAML
- 트리덴트-클러스터역할.YAML
- 트리덴트-디멘트발병 YAML
- '삼류 서비스.YAML'
- '삼중인대.YAML'
- 트라이덴트-서비스계정.YAML

이러한 파일을 생성한 후 필요에 따라 수정한 다음 '--use-custom-YAML'을 사용하여 사용자 지정 배포를 설치할 수 있습니다.

```
./tridentctl install -n trident --use-custom-yaml
```

다음 단계

Astra Trident를 구축한 후 백엔드 생성, 스토리지 클래스 생성, 볼륨 프로비저닝 및 POD에 볼륨 마운팅으로 진행할 수 있습니다.

1단계: 백엔드를 생성합니다

이제 Astra Trident에서 볼륨을 프로비저닝하는 데 사용할 백엔드를 만들 수 있습니다. 이렇게 하려면 필요한 매개변수가 포함된 백엔드 .json 파일을 만듭니다. 서로 다른 백엔드 유형에 대한 샘플 구성 파일은 'ample-input' 디렉토리에서 찾을 수 있습니다.

을 참조하십시오 "여기" 백엔드 유형에 맞게 파일을 구성하는 방법에 대한 자세한 내용은 을 참조하십시오.

```
cp sample-input/<backend template>.json backend.json
vi backend.json
```

```
./tridentctl -n trident create backend -f backend.json
+-------------+----------------+--------------------------------------
+--------+---------+
|    NAME     | STORAGE DRIVER |                  UUID                  |
STATE   | VOLUMES |
+-------------+----------------+--------------------------------------
+--------+---------+
| nas-backend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       0 |
+-------------+----------------+--------------------------------------
+--------+---------+
```

생성에 실패하면 백엔드 구성에 문제가 있는 것입니다. 다음 명령을 실행하여 로그를 보고 원인을 확인할 수 있습니다.

```
./tridentctl -n trident logs
```

문제를 해결한 후 이 단계의 처음으로 돌아가서 다시 시도하십시오. 자세한 문제 해결 팁은 를 참조하십시오 "문제 해결" 섹션을 참조하십시오.

2단계: 스토리지 클래스를 생성합니다

Kubernetes 사용자는 가 지정된 영구 PVC(Volume Claim)를 사용하여 볼륨을 프로비저닝합니다 "스토리지 클래스" 이름별. 세부 정보는 사용자로부터 숨겨지지만 스토리지 클래스는 해당 클래스에 사용되는 공급자(이 경우 Trident)와 해당 클래스가 프로비저닝자로부터 의미하는 바를 식별합니다.

Kubernetes 스토리지 클래스를 생성할 때 볼륨을 지정할 시기를 지정하십시오. 수업 구성에서는 이전 단계에서 생성한 백엔드를 모델링해야 하므로 Astra Trident가 이를 사용하여 새 볼륨을 프로비저닝합니다.

가장 간단한 스토리지 클래스는 설치 프로그램과 함께 제공되는 'ample-input/storage-class-csi.yAML.Templ' 파일을 기반으로 하는 것으로, 스토리지 드라이버 이름으로 *_backend_type*'을(를) 대체합니다.

```
./tridentctl -n trident get backend
+-------------+----------------+--------------------------------------
+--------+---------+
|    NAME     | STORAGE DRIVER |                 UUID                 |
STATE  | VOLUMES |
+-------------+----------------+--------------------------------------
+--------+---------+
| nas-backend | ontap-nas      | 98e19b74-aec7-4a3d-8dcf-128e5033b214 |
online |       0 |
+-------------+----------------+--------------------------------------
+--------+---------+

cp sample-input/storage-class-csi.yaml.templ sample-input/storage-class-
basic-csi.yaml

# Modify __BACKEND_TYPE__ with the storage driver field above (e.g.,
ontap-nas)
vi sample-input/storage-class-basic-csi.yaml
```

Kubernetes 오브젝트이므로 쿠버틀에서 생성되도록 kubtl을 사용하십시오.

```
kubectl create -f sample-input/storage-class-basic-csi.yaml
```

이제 Kubernetes 및 Astra Trident에 * basic-CSI * 스토리지 클래스가 표시됩니다. Astra Trident는 백엔드에서 풀을 검색했습니다.

```
kubectl get sc basic-csi
NAME         PROVISIONER             AGE
basic-csi    csi.trident.netapp.io   15h

./tridentctl -n trident get storageclass basic-csi -o json
{
  "items": [
    {
      "Config": {
        "version": "1",
        "name": "basic-csi",
        "attributes": {
          "backendType": "ontap-nas"
        },
        "storagePools": null,
        "additionalStoragePools": null
      },
      "storage": {
        "ontapnas_10.0.0.1": [
          "aggr1",
          "aggr2",
          "aggr3",
          "aggr4"
        ]
      }
    }
  ]
}
```

3단계: 첫 번째 볼륨을 프로비저닝합니다

이제 첫 번째 볼륨을 동적으로 프로비저닝할 준비가 되었습니다. 이 작업은 Kubernetes를 생성하여 수행합니다 "영구적 볼륨 클레임" (PVC) 개체.

방금 만든 저장소 클래스를 사용하는 볼륨에 대해 PVC를 생성합니다.

예를 들면 'Sample-input/PVC-basic-CSI.YAML'을 참조한다. 스토리지 클래스 이름이 생성한 이름과 일치하는지 확인합니다.

```
kubectl create -f sample-input/pvc-basic-csi.yaml

kubectl get pvc --watch
NAME      STATUS    VOLUME                                     CAPACITY
ACCESS MODES  STORAGECLASS   AGE
basic     Pending
basic          1s
basic     Pending   pvc-3acb0d1c-b1ae-11e9-8d9f-5254004dfdb7   0
basic          5s
basic     Bound     pvc-3acb0d1c-b1ae-11e9-8d9f-5254004dfdb7   1Gi
RWO           basic          7s
```

4단계: POD에 볼륨을 마운트합니다

이제 볼륨을 마운트하겠습니다. PV를 "/usr/share/nginx/html"에 장착하는 nginx 포드를 시작합니다.

```
cat << EOF > task-pv-pod.yaml
kind: Pod
apiVersion: v1
metadata:
  name: task-pv-pod
spec:
  volumes:
    - name: task-pv-storage
      persistentVolumeClaim:
       claimName: basic
  containers:
    - name: task-pv-container
      image: nginx
      ports:
        - containerPort: 80
          name: "http-server"
      volumeMounts:
        - mountPath: "/usr/share/nginx/html"
          name: task-pv-storage
EOF
kubectl create -f task-pv-pod.yaml
```

```
# Wait for the pod to start
kubectl get pod --watch

# Verify that the volume is mounted on /usr/share/nginx/html
kubectl exec -it task-pv-pod -- df -h /usr/share/nginx/html

# Delete the pod
kubectl delete pod task-pv-pod
```

이때 POD(애플리케이션)는 더 이상 존재하지 않지만 볼륨은 여전히 존재합니다. 원하는 경우 다른 포드에서 사용할 수 있습니다.

볼륨을 삭제하려면 클레임을 삭제합니다.

```
kubectl delete pvc basic
```

이제 다음과 같은 추가 작업을 수행할 수 있습니다.

- "추가 백엔드를 구성합니다."
- "추가 스토리지 클래스를 생성합니다."

저작권 정보

Copyright © 2022 NetApp, Inc. All Rights Reserved. 미국에서 인쇄된 본 문서의 어떠한 부분도 저작권 소유자의 사전 서면 승인 없이는 어떠한 형식이나 수단(복사, 녹음, 녹화 또는 전자 검색 시스템에 저장하는 것을 비롯한 그래픽, 전자적 또는 기계적 방법)으로도 복제될 수 없습니다.

NetApp이 저작권을 가진 자료에 있는 소프트웨어에는 아래의 라이센스와 고지사항이 적용됩니다.

본 소프트웨어는 NetApp에 의해 '있는 그대로' 제공되며 상품성 및 특정 목적에의 적합성에 대한 명시적 또는 묵시적 보증을 포함하여(이에 제한되지 않음) 어떠한 보증도 하지 않습니다. NetApp은 대체품 또는 대체 서비스의 조달, 사용 불능, 데이터 손실, 이익 손실, 영업 중단을 포함하여(이에 국한되지 않음), 이 소프트웨어의 사용으로 인해 발생하는 모든 직접 및 간접 손해, 우발적 손해, 특별 손해, 징벌적 손해, 결과적 손해의 발생에 대하여 그 발생 이유, 책임론, 계약 여부, 엄격한 책임, 불법 행위(과실 또는 그렇지 않은 경우)와 관계없이 어떠한 책임도 지지 않으며, 이와 같은 손실의 발생 가능성이 통지되었다 하더라도 마찬가지입니다.

NetApp은 본 문서에 설명된 제품을 언제든지 예고 없이 변경할 권리를 보유합니다. NetApp은 NetApp의 명시적인 서면 동의를 받은 경우를 제외하고 본 문서에 설명된 제품을 사용하여 발생하는 어떠한 문제에도 책임을 지지 않습니다. 본 제품의 사용 또는 구매의 경우 NetApp에서는 어떠한 특허권, 상표권 또는 기타 지적 재산권이 적용되는 라이센스도 제공하지 않습니다.

본 설명서에 설명된 제품은 하나 이상의 미국 특허, 해외 특허 또는 출원 중인 특허로 보호됩니다.

제한적 권리 표시: 정부에 의한 사용, 복제 또는 공개에는 DFARS 252.227-7013(2014년 2월) 및 FAR 52.227-19(2007년 12월)의 기술 데이터-비상업적 품목에 대한 권리(Rights in Technical Data -Noncommercial Items) 조항의 하위 조항 (b)(3)에 설명된 제한사항이 적용됩니다.

여기에 포함된 데이터는 상업용 제품 및/또는 상업용 서비스(FAR 2.101에 정의)에 해당하며 NetApp, Inc.의 독점 자산입니다. 본 계약에 따라 제공되는 모든 NetApp 기술 데이터 및 컴퓨터 소프트웨어는 본질적으로 상업용이며 개인 비용만으로 개발되었습니다. 미국 정부는 데이터가 제공된 미국 계약과 관련하여 해당 계약을 지원하는 데에만 데이터에 대한 전 세계적으로 비독점적이고 양도할 수 없으며 재사용이 불가능하며 취소 불가능한 라이센스를 제한적으로 가집니다. 여기에 제공된 경우를 제외하고 NetApp, Inc.의 사전 서면 승인 없이는 이 데이터를 사용, 공개, 재생산, 수정, 수행 또는 표시할 수 없습니다. 미국 국방부에 대한 정부 라이센스는 DFARS 조항 252.227-7015(b)(2014년 2월)에 명시된 권한으로 제한됩니다.

상표 정보

NETAPP, NETAPP 로고 및 http://www.netapp.com/TM에 나열된 마크는 NetApp, Inc.의 상표입니다. 기타 회사 및 제품 이름은 해당 소유자의 상표일 수 있습니다.